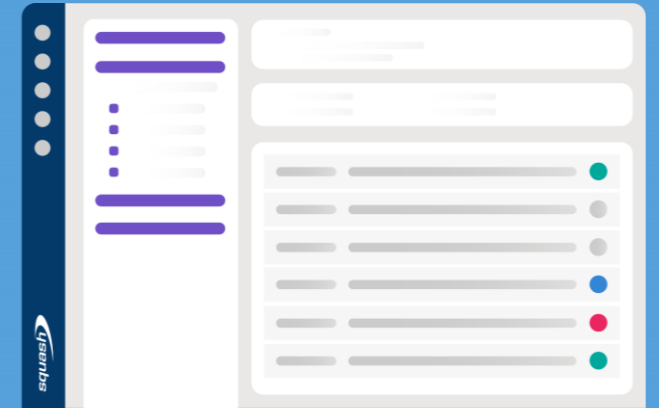


Les fondamentaux du test fonctionnel



Présentation générale de Squash TM

1. La Suite Squash
2. Squash TM

1. La suite Squash



*« Si l'homme ne façonne pas ses outils,
les outils le façonneront. »*

Arthur Miller



- Pourquoi un outil de gestion du patrimoine de test ?



Pourquoi un outil de gestion du patrimoine de test ?

- Capitaliser les expériences
- Gérer les étapes du projet : conception, exécution, pilotage
- Éditer des tableaux de bord à partir des indicateurs



Pourquoi un outil de gestion du patrimoine de test ?

- Capitaliser les expériences
 - Gérer les étapes du projet : conception, exécution, pilotage
 - Éditer des tableaux de bord
-
- Quels sont les avantages par rapport à Excel / Word ?



Pourquoi un outil de gestion du patrimoine de test ?

- Capitaliser les expériences
 - Gérer les étapes du projet : conception, exécution, pilotage
 - Éditer des tableaux de bord
-
- Quels sont les avantages par rapport à Excel / Word ?
 - Partager les informations
 - Travailler en équipe sur le même projet
 - Réutiliser plusieurs fois les mêmes composants du projet



- Outil de Gestion du patrimoine de test
- Outil Open Source inscrit au SILL*
- Utilisé par de nombreuses sociétés : Orange, RATP, MMA, BRED, Crédit Agricole...
- NB : Il existe bien d'autres outils sur le marché : Quality Center (HP), Testlink, etc.

*SILL : Socle Interministériel des Logiciels Libres ([Consulter le catalogue](#))



La suite Squash (Software QUality ASSurance enHancement) est un projet **Open Source** initié par HENIX, qui a pour objet la **structuration et l'industrialisation des tests fonctionnels**.

Le périmètre du projet **Squash** recouvre différents domaines de la qualification logicielle :



Squash TM est le gestionnaire de référentiel de tests.

- Référentiel **multi-projets**, versionné
- Gestion des **exigences**
- Gestion des **cas de test**
- Gestion des **campagnes** et **exécution** des tests
- **Déclaration des anomalies** dans un outil de gestion des anomalies
- **Pilotage** et **reporting**
- **Travail collaboratif**



Outillage d'**automatisation des tests** fonctionnels et d'industrialisation de leurs exécutions.

- Approche BDD des tests
- Workflow d'automatisation
- Plan de test automatisé
- Piloter l'exécution automatisé
- Résultats dans un rapports simplifié
- Interfaçage avec de nombreux outils d'automatisation (Cucumber, Cypress, Robot FRMK, Junit, SoapUI, UFT...)



Architecture micro-service permettant d'étendre les possibilités du pipeline CI/CD* pour faire des **tests en continu**.

- Test en continu depuis un pipeline
- Évolution facilitée de l'architecture du pipeline
- Récupération du plan d'exécution de Squash TM
- Outils de reporting
- Interfaçage avec des outils tiers (Jenkins, GitLab, Azure DevOps...)

* "CI/CD" signifie "Continuous Integration/Continuous Deployment" en anglais, ce qui se traduit en français par "Intégration Continue/Déploiement Continu".

2. Squash TM

Squash TM propose 7 espaces de travail

À chaque espace sa couleur ;-)

Espace Administration 

Espace Exigences 

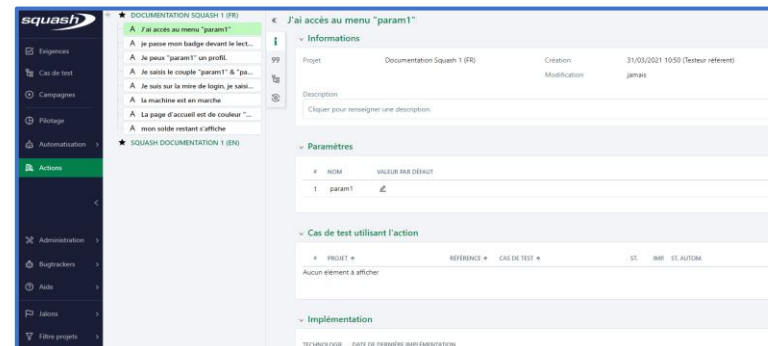
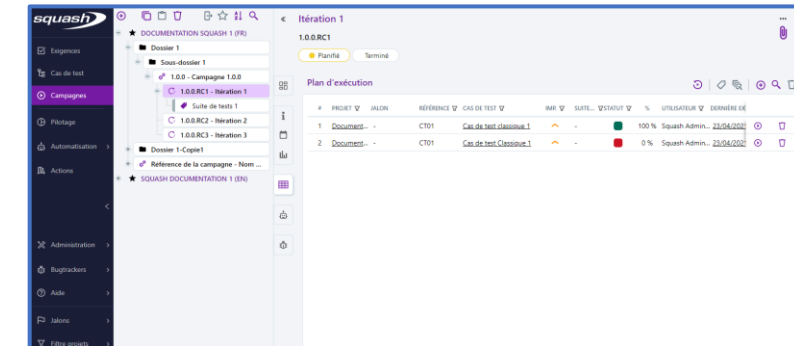
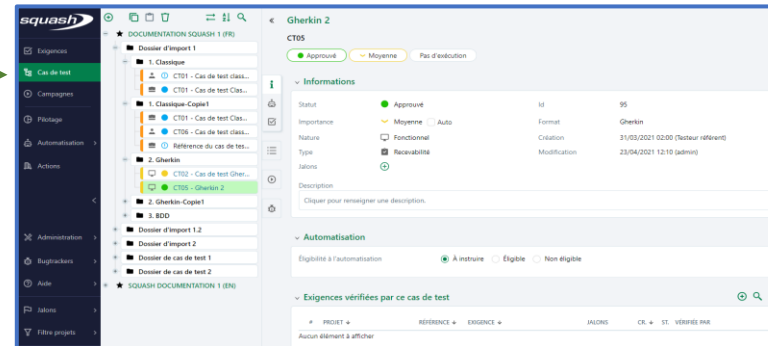
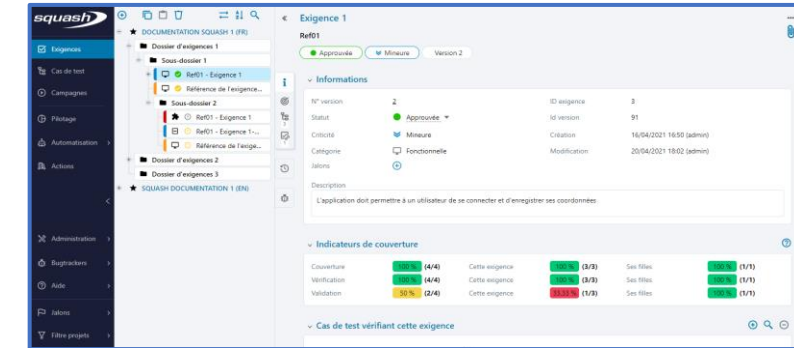
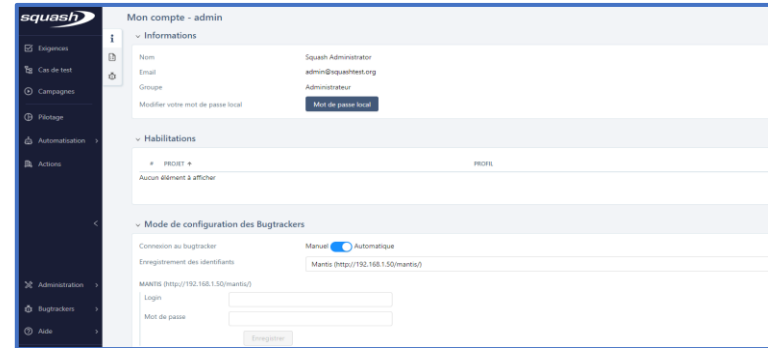
Espace Cas de tests 

Espace Campagnes 

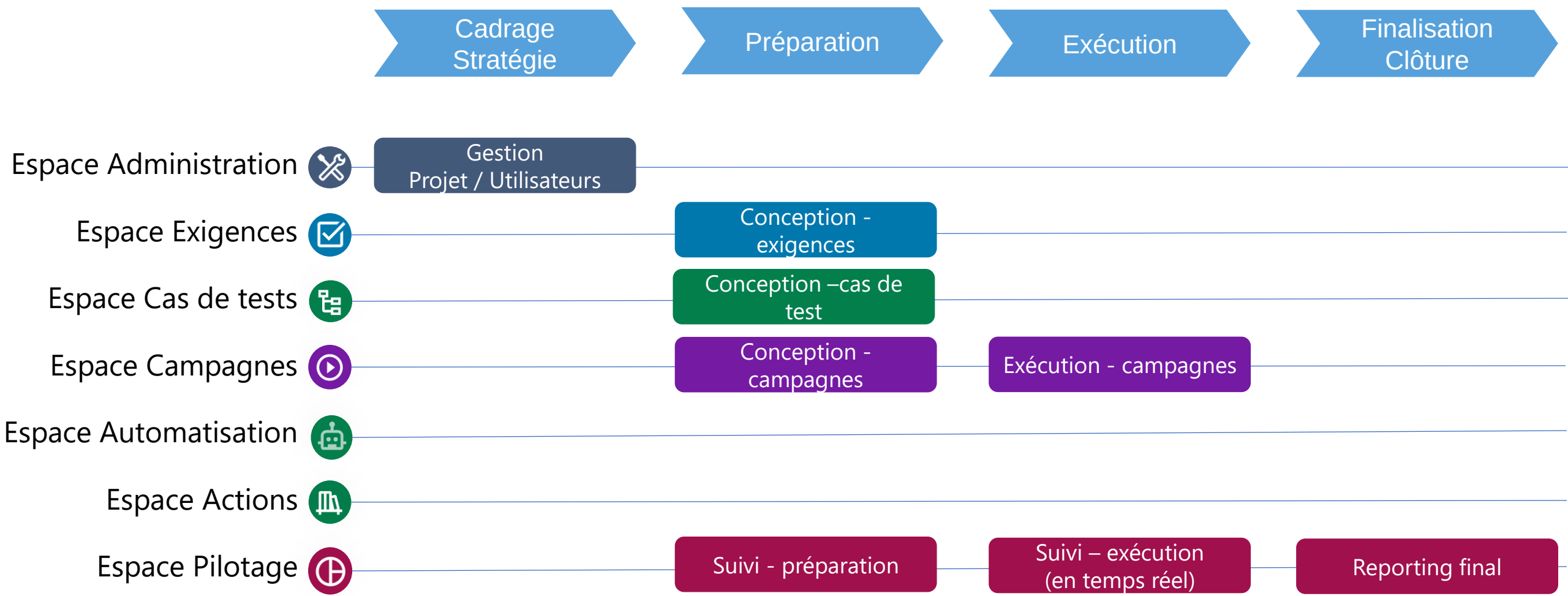
Espace Automatisation 

Espace Actions 

Espace Pilotage 



- Pendant la Recette, les 7 espaces de Squash seront utilisés en fonction des activités de test.



Espace Administration		(<i>Chef de Projet Recette</i>) Gestion Projets; Gestion et affectation des utilisateurs.
Espace Exigences		Gestion des exigences : Création des exigences; Qualification des exigences; Association de cas de test aux exigences; Recherche dans le patrimoine d'exigences.
Espace Cas de tests		Gestion des cas de test : Création des cas de test; Création des pas de test; Qualification des cas de test; Association d'une exigence à un cas de test; Recherche dans le patrimoine des cas de test; Suivi de la couverture des exigences par les tests.
Espace Campagnes		Gestion des campagnes et des itérations : Création d'une campagne de tests; Rattachement d'un cas de test à une campagne; Affectation d'une suite de test à un cas de test; Exécution des cas de test; Déclaration des fiches d'anomalie; Rattachement d'une fiche anomalie à un cas/pas de test.
Espace Pilotage		Pilotage : Cahier des Exigences/Cahier des Tests; Suivi de l'exécution des tests/Couverture qualitative.



- **Prendre en main l'application :**
 - Accéder à Squash TM
 - Comment s'identifier
 - Comment filtrer les projets sur lesquels travailler
- Naviguer dans les zones de SQUASH TM qui correspondent aux phases macros de la recette :
 - Espace Exigences
 - Espace Cas de test
 - Espace Campagnes
 - Espace Pilotage

Le patrimoine d'Exigences

1. La notion d'exigence
2. Identifier une exigence
3. Formaliser une exigence
4. Qualifier une exigence

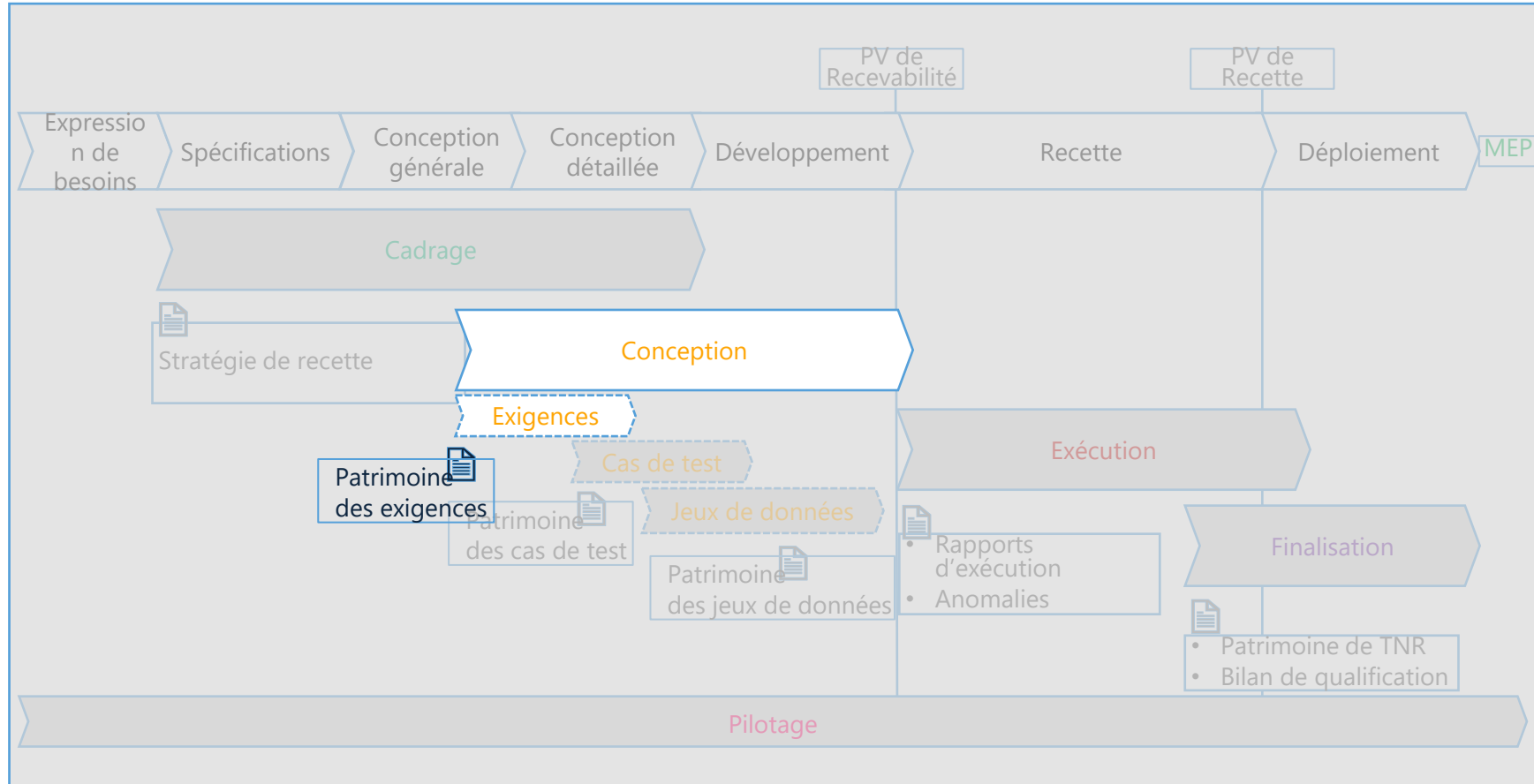
1. La notion d'exigence (condition de test)



« Ce que l'on conçoit bien s'énonce clairement, et les mots pour le dire arrivent aisément.. »

Nicolas Boileau – extrait de « L'Art poétique »

La définition du **niveau de Qualité** s'effectue durant la phase de conception, lors de la **formalisation des exigences**.



- Ce sont les **attentes** qui déterminent les **exigences** :



LES EXIGENCES ANALYSENT LES SPÉCIFICATIONS ET LES FONCTIONNALITÉS DU SYSTÈME

- Définitions ISTQB :



Exigence :

IEEE 610 : Condition ou capacité requise par un utilisateur pour résoudre un problème ou atteindre un objectif qui doit être tenu ou possédé par un système ou composant pour satisfaire à un contrat, standard, spécification ou autre document imposé formellement).

ISO 24765 : Une stipulation qui énonce des critères à satisfaire.

L'évolution du métier de testeur entraîne des évolutions dans les termes et définitions à utiliser.

Ce qu'on appelle « les exigences » dans la pratique (et outils de support de tests) est aussi dénommé **conditions de tests** dans le glossaire ISTQB.

L'ensemble des exigences/conditions de test formera le **Cahier d'exigences**, et constituera la **base de test** permettant d'élaborer les cas de test formalisés dans le **Cahier de Test**.

L'ensemble **Cahier d'exigences** + **Cahier de Test** constituera le **Cahier de Recette**.



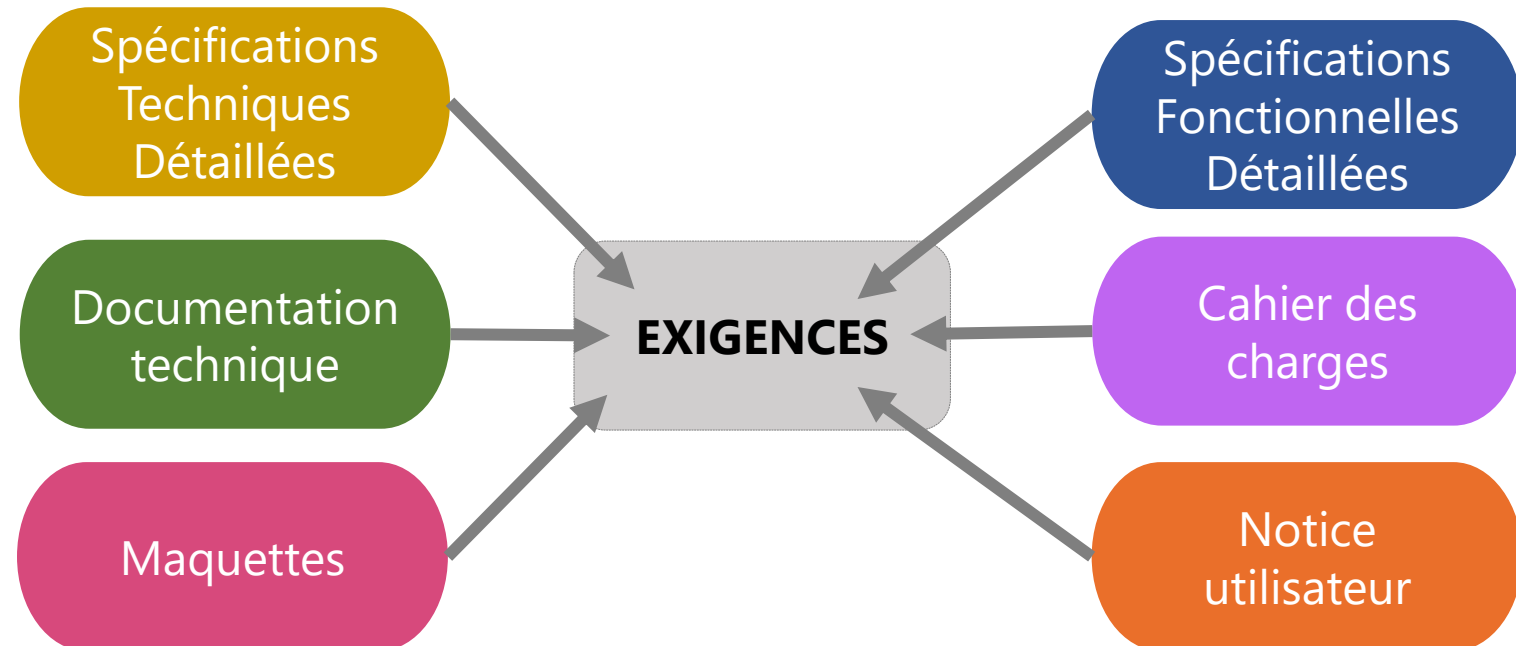
2. Identifier une exigence



Extraire et raffiner les exigences de test est un **processus itératif** qui consiste à **identifier** à partir de la documentation disponible tous les **cas d'utilisation et les règles qui les régissent**.

On analysera notamment tout ce qui concerne **l'IHM, les acteurs, la description des manipulations et des traitements de l'application**.

Pour **extraire et raffiner les exigences**, on prend en input tous les **documents formalisés** disponibles :

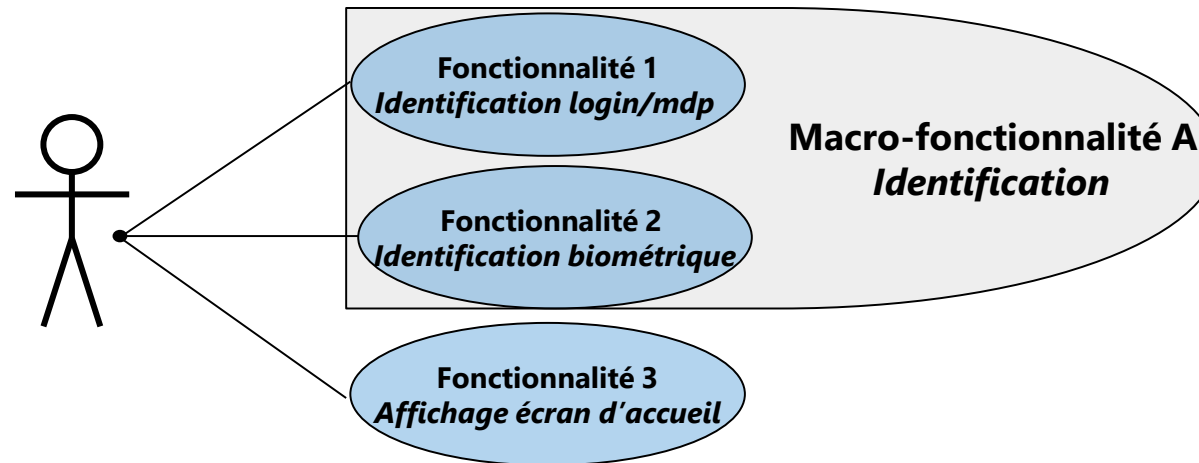




Un **cas d'utilisation** est une **description** narrative, une modélisation ou un scénario qui représente une **interaction spécifique** entre un utilisateur (ou un acteur) et un système logiciel.

Les cas d'utilisation aident à **définir les fonctionnalités, les interactions et les flux de travail** du logiciel d'une manière compréhensible et structurée.

Les **acteurs** sont les parties externes qui interagissent avec le système. Ils peuvent être des utilisateurs humains, d'autres systèmes, des périphériques, etc.



On identifiera d'abord les groupes fonctionnels, les macro-fonctionnalités. Pour faciliter l'analyse, **toujours aller du général vers le particulier, du nominal vers les comportements alternatifs** (exceptions, messages d'erreur...)



Identifier des exigences

On considère le système de gestion d'un DAB (Distributeur Automatique de Billets).

Les Spécifications du SUT indiquent que :

- Le distributeur délivre de l'argent à tout porteur d'une carte de la banque X. (autorisation d'un certain montant par le Système d'Information de la banque) ou d'une carte VISA (autorisation à distance par le Système d'autorisation VISA).
- Pour les clients de la banque X, il permet :
 - La consultation du solde du compte
 - Le dépôt d'argent (chèque ou numéraire)
- Toute transaction est sécurisée et nécessite une authentification (code personnel vérifié par le code enregistré sur la puce de la carte – la carte est avalée après trois échec).
- Un opérateur de maintenance se charge d'allumer ou d'éteindre le DAB. C'est la même personne qui recharge le distributeur.



Dans un premier temps, identifiez les acteurs, les fonctionnalités, les manipulations.
Dans un second temps, identifiez les actions et responsabilités des acteurs concernant les actions.
(En binômes ou à 3, Timebox : 10 à 15 mn)

3. Formaliser une exigence



- Pour chaque cas d'utilisation, on **identifiera et on rédigera** les exigences s'y rapportant.
- Mais les spécifications ne sont pas toujours adaptées à la pratique du test. Il faut **les transcrire, les reformuler et les adapter** pour les utiliser dans une démarche de qualité.
- Une exigence traduit **une règle de gestion** ou **un comportement attendu du système**.
- La rédaction d'une exigence implique **l'exactitude, la précision et l'absence de toute ambiguïté**.
- L'utilisation d'un modèle/template facilite leur rédaction :

Acteur	Action	Contexte (optionnel)
LE SYSTÈME L'UTILISATEUR }	DOIT [FAIRE CECI]	[QUAND CELA]

Par exemple : « L'utilisateur doit renseigner le champ « date » au format JJ/MM/AAAA. »

« Le système doit afficher le message d'erreur « [libellé] » en cas de mot de passe erroné. »

4. Qualifier une exigence



*Importance
métier de la
fonctionnalité*

Quand les exigences de test ont été formalisées, il faut :

- Les qualifier en leur attribuant une « **importance** », appelée **criticité**;
- La criticité d'une **exigence non encore évaluée** sera : **Non définie**.
- Les niveaux de criticité habituels d'une exigence sont : **Critique / Majeure / Mineure**;
- La criticité sera évaluée en fonction de **2 critères : l'importance métier de la fonctionnalité** et **sa fréquence d'utilisation** en s'aidant de la matrice suivante :

	<i>Fréquence d'utilisation de la fonctionnalité</i>		
	Très fréquent	Fréquent	Rare
Très important	Critique	Critique	Majeure
Important	Critique	Majeure	Mineure
Peu important	Majeure	Mineure	Mineure



Une dernière étape reste à franchir, pour s'assurer de la complétude et de l'exactitude du référentiel d'exigences constitué :

- Sa **validation par la MOA (le Métier)**;
- Elle permettra, avant de constituer le référentiel de cas de test, de confirmer que **le périmètre à tester est bien couvert par les exigences** (les règles que la MOA souhaite voir vérifier) et que **ces règles ont été correctement comprises et rédigées** par les testeurs.
- Le processus formel de validation par la MOA se manifeste par le **statut des exigences** :
 - **En cours de rédaction** (pas encore finalisée par le testeur)
 - **À valider** (finalisée par le testeur, et candidate à la validation par la MOA)
 - **Validée** (= à tester, on peut créer le(s) cas de test)
 - **Obsolète** : Devenue non applicable pour la version en cours du SUT



En fonction des évolutions et changements des différentes versions du SUT, les exigences peuvent évoluer.

Pour des raisons de traçabilité, elles seront versionnées, et les versions antérieures identifiées et archivées dans le gestionnaire de tests.

La version à prendre en compte est toujours la dernière version en cours.

- La nouvelle version devra suivre le même processus de validation.

Par exemple, pour cette exigence :

v.1 : « Après identification de l'utilisateur par login/mdp, la page d'accueil doit être affichée. »

➡ **Modification mineure**, sans changement du sens => pas de versionnage.

v.1 : « Après identification de l'utilisateur par login/mdp, au clic sur le bouton « ENTRER », la page d'accueil doit être affichée. »

➡ **Évolution** : La page affichée ne sera plus la page d'accueil, mais le tableau de bord de l'utilisateur.

➡ **Modification majeure** => l'exigence revue passe en v.2, la v.1 est archivée et reste consultable.

v.2 : « Après identification de l'utilisateur par login/mdp, au clic sur le bouton « ENTRER », la page du tableau de bord spécifique à l'utilisateur doit être affichée. »

- **Le référentiel d'exigences (Cahier d'exigences) doit remplir les critères suivants :**
 - Une exigence est **toujours un comportement attendu** : « *Le logiciel doit (...)* »
 - Une **langue précise** : un glossaire identique pour toutes les versions, les chapitres, etc.
 - Un **langage partagé** : tous les rédacteurs doivent employer le même vocabulaire. Ne pas laisser de place à l'interprétation !
 - Un **référencement cohérent** : établir des références (ou conventions de nommage) qui évolueront facilement, qui sont utilisées partout et par tous. Par exemple ID_ADMIN_001 (Identification Administrateur exigence 1)
 - Une **organisation structurée** : du plus général au plus particulier par exemple (« top-down »). Réfléchir à l'organisation du patrimoine des exigences pour retrouver rapidement une information maintenant et dans la 10^{ème} version... Éviter plus de 3 niveaux d'indentation dans l'arborescence.
 - Une exigence doit être **testable** (elle doit offrir un moyen de prouver que le système satisfait à son énoncé).



Mantra du jour : *Une exigence doit pouvoir être comprise par n'importe qui* !*

* N'importe quel testeur, AMO, etc.

Tips & Tricks

- **Attention à la ponctuation**

« Le logiciel calcule la vitesse du missile et sa trajectoire, en 5 secondes maximum », a un sens différent de :
« Le logiciel calcule la vitesse du missile, et sa trajectoire en 5 secondes maximum ».

- **Attention au « raffinement » (niveau de précision de l'exigence)**

« Le logiciel calcule la trajectoire du missile en moins de 5 secondes »

=> On a ici **2 sujets** (classes) : fonctionnelle et performance. **Donc 2 exigences DISTINCTES :**

- « Le logiciel calcule la trajectoire du missile » (fonctionnelle)
- « Le calcul de la trajectoire du missile est effectué en moins de 5 secondes » (non fonctionnelle : performance).

- **Attention à la précision**

« L'application doit être intégrée aux systèmes de paiement » : **Imprécise !**

L'exigence doit être rédigée ainsi : « L'application devra accepter les systèmes de paiements Visa, PayPal et Virement bancaire ».

- **Attention à la cohérence**

Deux exigences ne doivent pas se contredire.

« L'entrée A apparaît seulement lorsque l'entrée B apparaît ». Et 10 pages plus loin : « L'entrée B doit commencer 15 secondes après l'apparition de l'entrée A ».

- **Attention à la redondance**

À deux endroits différents du CDC/de la spécification :

« Le retrait maximum est de 1 800 € » et « Le montant de tout retrait est compris entre 100 € et 1 800 € ».

Les principaux défauts constatés dans la rédaction des exigences sont les suivants :

- le **bruit** : information non pertinente, voire inutile ;
→ **Pas de bla-bla**
- le **remords** : forme de bruit dû à la modification du texte a posteriori ;
→ **Faites-vous confiance**
- le **silence** : absence d'informations ou notions non explicitées ;
→ **Mettez-vous à la place d'un néophyte**
- la **sur-spécification** : en dire trop sur la solution (dérive vers la conception) ;
→ **Une exigence exprime le « QUOI », pas le « COMMENT »**
- l'**incohérence** : contradiction interne ou exigences incompatibles ;
→ **Faites appel à la revue par un pair en cas de doute**
- l'**ambiguïté** : termes imprécis avec plusieurs interprétations possibles ;
→ **Utilisez un vocabulaire précis et factuel**
- l'**exigence intestable** : vœu pieux ;
- la **redondance**, le **non-respect du standard**, les **exigences absentes...**

En cas de doute, CREUSEZ ! Équipe, CP Recette ou MOA pourront vous apporter la réponse.



Photofiltre : Créer des exigences

- Dans l'espace **Exigences de Squash**, rédigez les exigences de test de l'application **PhotoFiltre à partir du Manuel utilisateur. (v. Moodle)**
 - 1^{ère} étape : lire le *Manuel utilisateur*.
 - 2^{ème} étape : identifier les fonctionnalités.
 - 3^{ème} étape : extraire, rédiger et qualifier les exigences de test.
- **Attention à respecter les critères suivants :**
 - Le **périmètre** : seulement le *Manuel utilisateur* même s'il renvoie à l'application.
 - **L'organisation** des exigences dans l'arbre : par fonctionnalité, par objet métier, une partie réservée à l'interface, etc.
 - La **rédaction** d'une exigence :
 - « *Le logiciel doit* permettre à l'utilisateur **d'ajouter** du bruit sur une photo. »
 - « *L'interface doit* **afficher** les menus suivants ... »
 - « *Le logiciel doit* permettre à l'utilisateur **d'appliquer** un masque. »
 - La **qualification** des exigences (bonus de l'exercice) :
« *Le logiciel doit* permettre d'appliquer un filtre » : **critique**.

Le référentiel des cas de test

1. La notion de cas de test
2. La notion de jeu de données
3. Technique de conception des tests
4. Identifier les tests de non-régression
5. La couverture fonctionnelle

1. La notion de cas de test

Chaque exigence de test/condition de test **validée** (par la MOA ou le PO) doit être **vérifiée** par **un ou plusieurs cas de test**.

Les cas de test seront rédigés selon deux axes :

- Du **nominal** (tout se passe bien) aux **alternatives** (chemins, comportements alternatifs ou en erreur)
- Du **haut niveau** (cas de test abstrait, sans préciser exactement les données d'entrée et de sortie) vers le **niveau détaillé/bas niveau** (avec des valeurs précises)

Par exemple : Exigence à tester : « *L'application devra accepter les systèmes de paiements par Visa, PayPal et Virement bancaire* ».

CT Haut niveau (nominaux)	CT Bas niveau (nominaux)
Effectuer une commande et régler avec une carte Visa	Procédure détaillée « Effectuer une commande » + régler avec une carte Visa (+ <i>Données personnelles acheteur, Données article commandé, Données carte Visa (Nom / Date d'expiration / Numéro / Cryptogramme)</i> + Résultat(s) attendu(s)
Effectuer une commande et régler avec un compte Paypal	Procédure détaillée « Effectuer une commande » + régler avec un compte Paypal (+ <i>Données personnelles acheteur, Données article commandé, identifiant + mot de passe Paypal</i>) + Résultat(s) attendu(s)
Effectuer une commande et régler avec Virement bancaire	Procédure détaillée « Effectuer une commande » + régler par virement bancaire (+ <i>Données personnelles acheteur, compte bancaire acheteur, compte bancaire vendeur</i>) + Résultat(s) attendu(s)



Après ces cas de test nominaux, quelles idées auriez-vous pour des cas de test alternatifs ?

Cas de test :



« Ensemble de **valeurs d'entrée**, de **pré-conditions d'exécution**, de **résultats attendus** et de **post-conditions d'exécution**, développé pour un objectif ou une condition de test particulier, tel qu'**exécuter un chemin particulier** d'un programme ou **vérifier le respect d'une exigence** spécifique ».

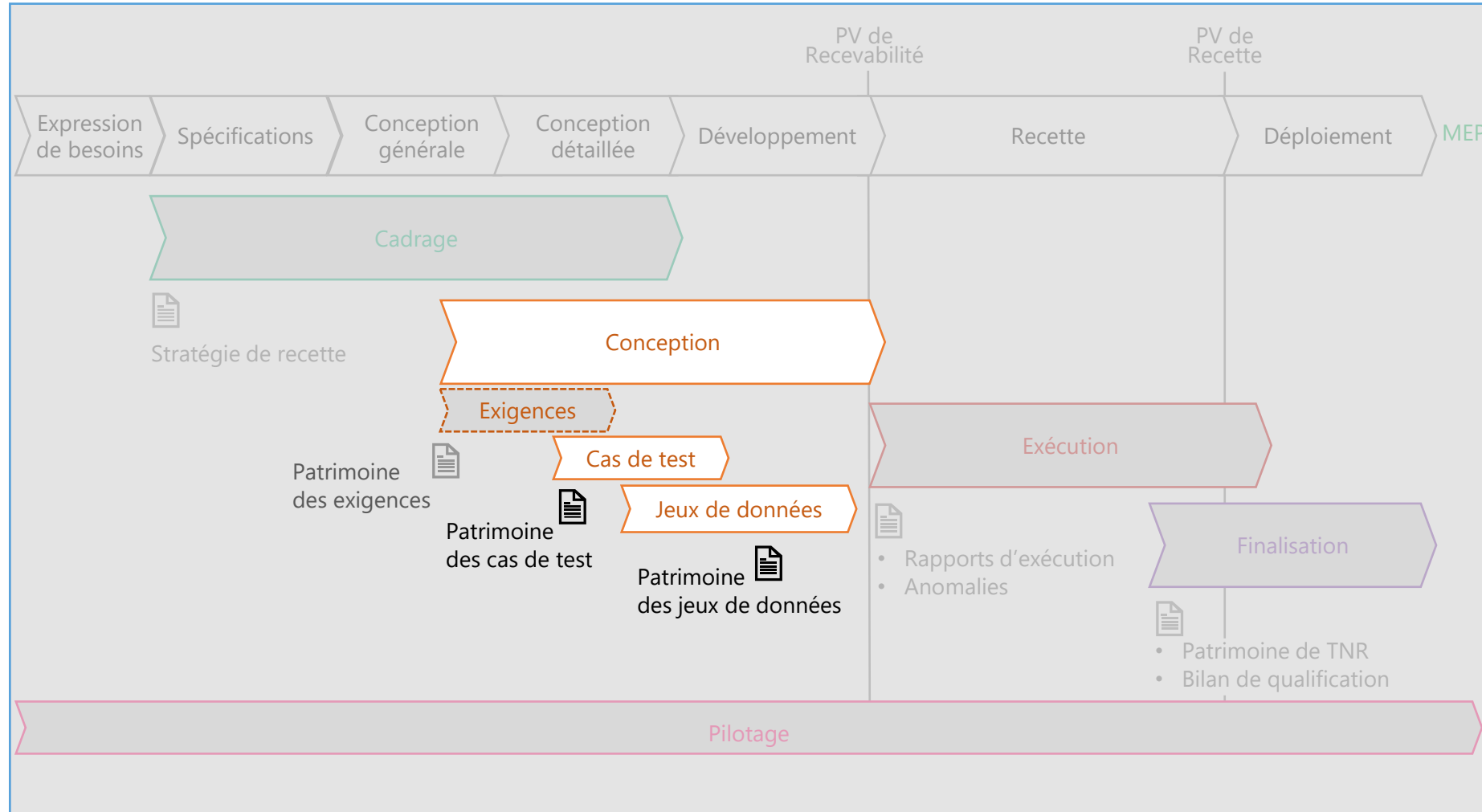
Les cas de tests vont donc définir des conditions et un chemin spécifique afin de vérifier le niveau de qualité attendu.

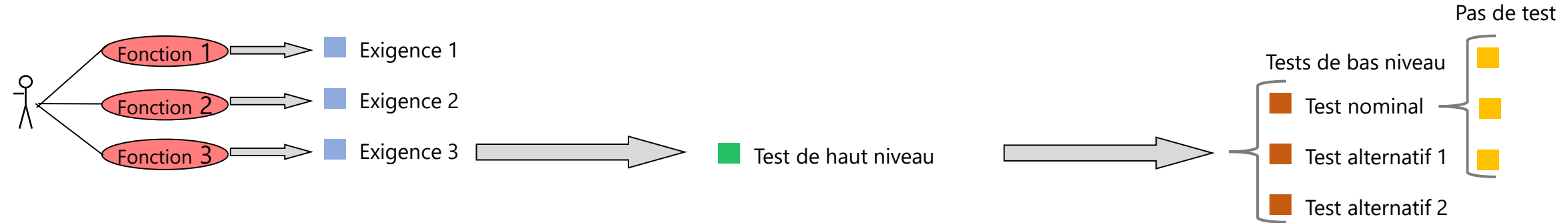
Tests basés sur les exigences :



Approche des tests où les cas de tests sont conçus sur la base des **objectifs de tests et conditions de tests déduites des exigences**, par ex. tests portant sur des **fonctions spécifiques** ou examinant des **attributs non-fonctionnels** tels la **fiabilité** ou **l'utilisabilité**.

La **définition des résultats attendus** s'effectue durant la phase de conception des cas de test, avec la formalisation des cas de test et des jeux de données.





Fonctionnalité

Exigence

Test de haut niveau

Cas de test de bas niveau (détaillé)

Connexion à l'application

L'application doit permettre une connexion avec un login et un mot de passe valide

Etant donné que je suis sur la page de connexion
Quand je rentre un mot de passe dans le champ « Mot de passe »
Et un login dans le champ « login »
Et que je clique sur le bouton [VALIDER]
Alors j'arrive sur la page d'accueil

Actions

Se connecter à la page https://...

Saisir le login « **Fabien** » dans le champ login

Saisir le mot de passe « **1234** » dans le champ mot de passe

Cliquer sur le bouton **[VALIDER]**

Résultats attendus

La page de connexion s'affiche

Le login « **Fabien** » est affiché

Le champ mot de passe affiche « ******** »

La page d'accueil du site est affichée

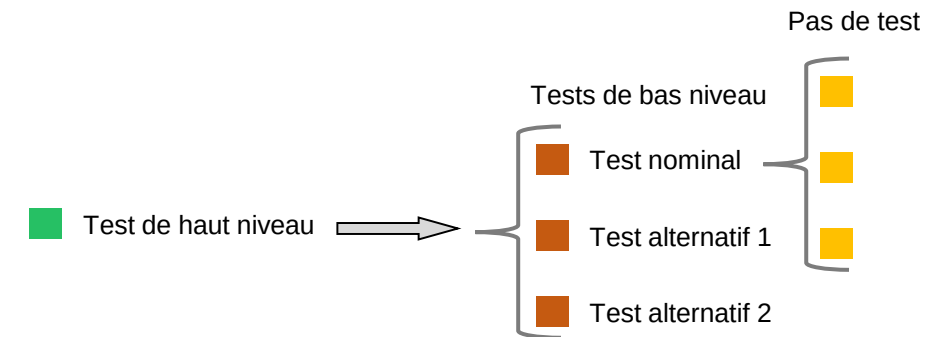
Caractéristiques des cas de test :

- Différents **types** de cas de test :
 - Cas nominal / alternatif
 - Cas passant / non passant
- Le « **Cas de test** » se compose de « **Pas de test** » :
 - Le test se déroule pas à pas
En règle générale :
→ un Cas de test comporte entre 10 et 20 Pas de test.
- Le cas de test se termine toujours par un **résultat attendu** :
 - A la fin du test, « La page d'accueil du site est affichée ».
- **Attention à la rédaction !**
 - Se défier du copier/coller !
 - Les **actions** se rédigent avec **un verbe à l'infinitif**.



Mantra du jour : *Un Cas de test doit pouvoir être compris par n'importe qui* !*

* N'importe quel testeur...



Cas de test

Actions

Se connecter à la page
https://...

Saisir le login « **Fabien** »
dans le champ login

Saisir le mot de passe
« **1234** » dans le champ
mot de passe

Cliquer sur le bouton
[VALIDER]

Résultats attendus

La page de connexion
s'affiche

Le login « **Fabien** » est
affiché

Le champ mot de passe
affiche « ******** »

La page d'accueil du site est
affichée

Exemple de différents types cas de test de bas niveau pour un même test de haut niveau :

Test haut niveau : « Vérifier la connexion (passante et non-passante) avec login + mot de passe »

- Test nominal passant : avec un login et un mdp correct

Résultat attendu → la connexion fonctionne

- Test non-passant 1 : avec Login inconnu

Résultat attendu → La connexion est refusée avec le message « *Login inconnu* »

- Test non-passant 2 : avec MdP incorrect

Résultat attendu → La connexion est refusée avec le message « *MdP incorrect* »

- Test non-passant 3 : Sans renseigner aucun champ

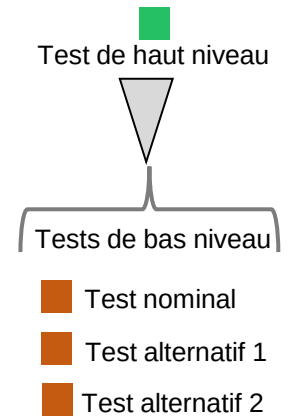
Résultat attendu → On renvoie le message « *Veuillez renseigner les champs* »

- Test alternatif passant 1 : avec un login/mdp correct avec des caractères spéciaux (ex: ê, é, è, ë...)

Résultat attendu → la connexion fonctionne

- Test alternatif passant 2 : avec un login/mdp correct avec beaucoup de caractères (ex:144 caractères)

Résultat attendu → la connexion fonctionne



Tips & Tricks

Les cas de test ci-contre sont fondés sur les exigences extraites des SFD.

Saurez-vous identifier un problème de sécurité potentiel qui devrait être signalé à la MOA ?

Nommer et référencer le cas de test

- Le **nom** du test doit permettre de **comprendre l'objet testé**, voire le **résultat attendu**. Il doit être **unique** et **précis**. Cela aide à savoir ce qui est déjà testé sur le périmètre, ainsi que pour retrouver les tests.
Par exemple, plutôt que « Vérifier la connexion », vous pouvez libeller votre cas de test « Vérifier la connexion en échec pour un compte verrouillé ».
- La **référence** fait l'objet d'une nomenclature qui reprend l'arborescence de l'espace « Cas de test » (cf slides « Bonnes pratiques transverses aux espaces »).
- Eviter de négliger la **description** du test. Celle-ci permet de **clarifier** ce qui est vérifié par le test. Lorsqu'il n'y a pas de description, la compréhension des objets testés doit se faire à partir des étapes du test, ce qui est laborieux et source de confusion. La description **fluidifie** l'exécution du test et sa mise à jour, surtout si différentes personnes sont amenées à travailler sur ces phases.

Qualifier le cas de test

- La **nature et le type** du test permettent d'avoir de la visibilité sur la répartition des tests du patrimoine et son évolution. Elles permettent également de rechercher plus facilement des tests qui serviront à alimenter des plans de tests (ex. plan de test d'accessibilité, de non régression, ...).
- Le **type** du test suivra son cycle de vie. Les questions à se poser dépendent notamment de ce que le test vérifie :
 - Lorsque j'écris un cas de test qui vérifie une évolution, je le type « **Evolution** »,
 - Le cas de test a été exécuté en succès, et il est éligible à entrer dans le patrimoine de non régression ?
Oui, alors je le type en « **Non régression** »,
 - Le cas de test a-t-il permis de détecter une anomalie? Oui, alors je le type en **correctif**...

Qualifier le cas de test

- Le **statut** du test indique si le contenu du test est **fiable** et **exécutable** en l'état, et il doit donc être maintenu tout au long du cycle de vie du système à tester :
 - À sa création, le cas de test est au statut « **En cours de rédaction** »
 - Une fois qu'il est rédigé, vous pouvez le passer au statut « **A approuver** » et mettre en place un workflow de validation auprès d'une MOA, ou d'une relecture par des paires qui le passeront au statut « **Approuvé** ».
 - Tout cas de test qui nécessite une modification devra être mis au statut « **A mettre à jour** » afin d'être facilement retrouvé lors des phases de conception de test. Ce ciblage peut avoir lieu après ou pendant une phase d'exécution des tests si la personne n'a pas la charge disponible pour mettre à jour le test en parallèle de son exécution.
 - Un cas de test n'est plus valide lorsque son exigence est obsolète. Dans ce cas, le cas de test sera passé au statut « **Obsolète** ».

Qualifier le cas de test

- **L'importance** du test permettra notamment de **sélectionner les tests de plus haute importance pour couvrir les fonctionnalités les plus critiques** lors de l'estimation de l'effort de test.
- Elle sera également un critère de choix pour **prioriser l'exécution des tests**.
- Il est également possible d'avoir un **calcul automatique de l'importance du test**, qui se base sur la criticité des exigences reliées, en utilisant la coche du champ « Auto ».
- L'équipe de test peut se baser sur les questions de la **grille ci-après pour évaluer l'importance** du test.

La notion de cas de test

Questions	Importance	Exemples	Conséquences
- Est-ce que le cas teste l'utilisation nominale d'une fonctionnalité ? - Est-ce que le cas teste spécifiquement une fonctionnalité critique de l'application ? - Est-ce que si mon cas de test est en échec, cette fonctionnalité sera indisponible pour les utilisateurs ? - Est-ce que le cas de test peut être utilisé comme test de recevabilité ?	Très haute	- Tests de recevabilité - Tests de bout en bout - Cas nominaux	- Tests exécutés très fréquemment, voire à chaque campagne - Les anomalies liées seront souvent bloquantes
- Est-ce que le cas est passant ? - Est-ce que le cas teste spécifiquement une fonctionnalité critique de l'application ? - Est-ce que si mon cas de test est en échec, l'utilisateur disposera d'une solution alternative dans l'application ?	Haute	- Cas nominaux - Cas alternatifs passants	- Tests joués à chaque campagne impactant le périmètre concerné - Les anomalies liées seront souvent majeures
- Est-ce que le cas est non-passant ? - Est-ce que le cas a vocation à alimenter le patrimoine de TNR pour rejeu ?	Moyenne	Cas non passants importants (sur des parcours critiques de l'application)	- Tests exécutés lors de campagnes plus étendues - Les anomalies liées seront majeures ou mineures
- Est-ce que le cas est non-passant ? - Est-ce que le cas vérifie des RG d'ergonomie, d'affichage ? - Est-ce que le cas a été conçu spécifiquement pour vérifier un correctif ?	Faible	- Cas non passants - Vérifications d'affichage - Ergonomie	- Tests exécutés très peu fréquemment, uniquement dans des circonstances particulières (correction d'anomalie) ou campagnes de TNR complète - Les anomalies liées seront souvent mineures

Rédiger le cas de test

- Choisir des normes d'écriture des cas de test simplifie leurs compréhensions et leurs exécutions. Ces normes peuvent concerner par exemple, les profils, les boutons ou encore les messages.
- Exemples :
 - Les noms de profils utilisés doivent être en vert :
« Se connecter avec un profil **administrateur** »
 - Les boutons doivent être en gras entre crochet
« Cliquer sur le bouton **[Enregistrer]** »
 - Les messages doivent être en rouge et en italique. :
«Le message bloquant suivant s'affiche : «*Merci de renseigner tous les champs et de valider à nouveau*» »
 - Les variables et balises paramètres doivent être en turquoise. :
« Se connecter avec un profil **#{profil}** »

Liaison cas de test/exigence :

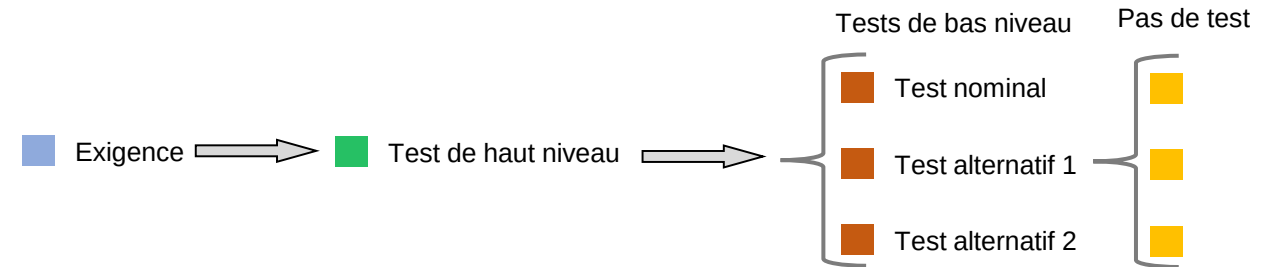
- L'**objectif** d'un cas de test est de **vérifier expérimentalement qu'une exigence est respectée** par le SUT. On créera donc un **lien (bijectif) entre les exigences et les cas de test** qui les vérifient.
- On **qualifiera l'importance des cas de test** en fonction de **la criticité des exigences** qu'il couvre.
- Différents **types de liens** exigences / cas de test :
 - Plusieurs cas pour valider 1 exigence
 - 1 cas valide plusieurs exigences
- Les cas de test de bas niveau (détaillés) peuvent être formalisés selon **deux niveaux** :
 - Le testeur **ne connaît pas** l'application → description très **didactique** :
 1. ouvrir le menu
 2. appuyer sur créer
 3. dans le champ « Nom », taper : « Dujardin »
 4. ...
 - Le testeur **connaît** l'application → description **productive** :
 1. créer un nouveau client « Jean Dujardin »
 2. ...



Dans quelles circonstances/types de mission ou SUT opter pour l'un ou l'autre niveau de rédaction ?



Rédigez les cas de test d'un DAB



Dans l'espace Cas de test de Squash, rédigez les différents cas de test pour le retrait, le RIB et la consultation de compte :

- **1^{ère} étape** : Reprendre les exigences rédigées précédemment;
- **2^{ème} étape** : pour chaque exigence, dérivez les différents tests de haut niveau nécessaires à sa vérification;
- **3^{ème} étape** : rédigez les différents cas de tests de bas niveau (Pas de test + données de test);
- **4^{ème} étape** : Pensez à bien qualifier les tests (statut, importance, nature, type, description).

2. La notion de jeu de données



« L'âge est une donnée biologique socialement manipulable et manipulée. »

Pierre Bourdieu

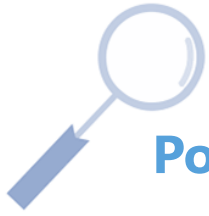
Scientifique, Sociologue (1930 - 2002)

« Les jeux de données en test logiciel sont comme les paires de chaussettes : il y en a toujours un qui s'avère incomplet quand on en a le plus besoin ! »

PAD



- Pourquoi est-il nécessaire de constituer des jeux de données ?



Pourquoi est-il nécessaire de constituer des jeux de données ?

- Chaque exécution des cas de test « consomme » des données, qui ne pourront peut-être pas être réutilisées (par ex. données « grillées » en base données, car modifiées/transformées)
- En cas d'incertitude ou d'hésitation dans la conclusion du CT ou en cas de demande de précisions du développeur, **le testeur doit être en mesure de réexécuter le cas de test** pour vérification.
- Plusieurs itérations étant exécutées, **le même cas de test sera exécuté à plusieurs reprises**.
- Le **plus coûteux** est de créer/trouver les **premières données** nécessaires pour un cas de test. Les données additionnelles seront construites suivant le **même modèle**.

Abondance de données ne nuit pas ! Soyez généreux, vous vous en remercirez plus tard ...



- Qu'est-ce qu'un jeu de données ?



Qu'est-ce qu'un jeu de données ?

- Un jeu de données (appelé aussi dataset, ou jeu d'essai) va permettre, pour chaque cas de test ayant pour objectif la validation d'une fonctionnalité de l'application, **d'affecter des valeurs** à la description d'une suite **d'actions** et de **résultats attendus**.

Les données sont décrites selon :

- Un **type** : *date, numérique, texte, alphanumérique, image, etc.*
Le **type de données** concerne la nature des données et détermine les opérations qui leur sont applicables.
- Un **format** : *jj/mm/aaaa, integer (nombre entier), jpeg, etc.*
Le **format de données** se réfère à la structure et à la représentation spécifiques dans lesquelles les données sont stockées ou échangées.
Il définit la manière dont les données sont organisées et codées pour être utilisées dans un contexte particulier.
- Une **taille** : 12 caractères maximum, etc.



Les types de données courants

- **Entier (Integer/INT)** : Ces données représentent des **nombre entiers sans partie décimale**, positifs ou négatifs.
- **Flottant (Floating-Point/FLOAT)** : Les nombres à virgule flottante sont utilisés pour représenter des **nombre réels**, positifs ou négatifs, y compris ceux avec une partie décimale.
- **Caractère (Character/CHAR)** : Les caractères représentent des **symboles, lettres, chiffres et signes de ponctuation**. Ils sont utilisés pour représenter du texte.
- **Chaîne de caractères (String)** : Une **séquence de caractères**, utilisée pour représenter du texte sous forme de phrases, mots, etc.
- **Booléens (Boolean/BOOL)** : Les valeurs booléennes représentent **le vrai (true) ou le faux (false)**, souvent utilisées pour les opérations logiques.
- **Tableau (Array)** : Les tableaux sont des **collections ordonnées d'éléments de même type**, accessibles par un index.
- **Liste (List) / Énumération (Enumeration)** : Les énumérations sont des types de données définis par l'utilisateur qui **énumèrent un ensemble de valeurs possibles**.
- **Données binaires (Binary Data)** : Utilisées pour stocker des **données brutes**, souvent utilisées pour les fichiers et les données non textuelles.
- **Données temporelles (Temporal Data)** : Utilisées pour stocker des **informations liées au temps**, telles que dates, heures ...



• Quel est l'objectif de la création des jeux de données ?

- Extraire ou créer/générer un **ensemble cohérent** de données correspondant à des critères fonctionnels sélectionnés, pour des applications développées ou en cours de développement.
- Réduire la **durée** et la **volumétrie** des tests.
- Garantir la **confidentialité** des données extraites (données anonymisées).

BONNES PRATIQUES :

- Extraire, créer, générer les jeux de données en amont (avant) l'exécution des tests.
- La réflexion sur les données peut commencer lors de la rédaction des exigences, dès qu'on en identifie les caractéristiques (type/format/longueur/modèles)

Modes de constitution des jeux de données :

- Par **construction** (saisie manuelle, scripts SQL, outils de génération de données).
- Par **extraction/copie** de données de production à partir des bases de données. Il faudra éventuellement anonymiser ces données.

BONNES PRATIQUES

- Prévoir suffisamment de données pour couvrir les besoins lors des différentes itérations.
- Un système permettant de 'réserver' les données peut s'avérer nécessaire afin d'éviter que les actions d'un testeur impactent les données et/ou les tests d'un autre testeur.



- Quels sont les avantages ?
- Quels sont les inconvénients ?



✓ Impact de la qualité du JDD sur la qualité de l'application

✓ Harmonisation de l'effort de test entre testeurs

✓ Permet de paralléliser les tests



✓ Effort de conception

✓ Attention aux bugs de JDD

✓ Effort de maintenance

Le jeu de données participe à la **couverture fonctionnelle** des tests :

- Si on dispose de **plusieurs environnements de test**, les tests se déroulent de façon autonome sans interférer (*p. ex. un SUT installé sur les postes de travail, chaque testeur disposant alors de sa propre BDD*).
- Sinon, lorsque la BDD est partagée, il faut **partitionner** les données pour les testeurs :
 - Grâce aux **jeux de données**
 - ALTEA AF (application qui permet de gérer les réservations de billets d'avion, passagers, vols.) : Les testeurs peuvent se répartir les données par **destination, par type de passager, etc.**
 - Application de gestion clients : Les testeurs se répartissent les données par ordre alphabétique du patronyme, ou par volume de données (Testeur 1 : les 100 premiers, etc.)
 - Par intervalle de **temps** :
 - le matin = les tests d'interface.
 - l'après-midi = les tests de gestion.
 - la nuit = les TNR automatisés.



Rédigez les jeux de données (Partie 1/2)

Sur la base des règles de gestion de l'application d'Air France *ALTEA* :

- Proposez des jeux de données de **types de vol** qui respectent le format attendu
- Proposez des jeux de données de **passagers** qui respectent le format attendu
- Proposez des jeux de données de **billets** qui respectent le format attendu
- Format de réponse : Excel.

Squash TM permet de **variabiliser** et de valoriser des pas de test par une gestion des paramètres et des jeux de données.

ACTION	RÉSULTAT ATTENDU
Dans la barre de recherche de `\${navigateur}` , entrer l' URL OK .	La mise de base de données d'OpenEcole s'affiche à l'écran

- 1 Onglet 'Pas de test' : au lieu de renseigner une valeur, renseigner un paramètre qui pourra prendre une ou plusieurs valeurs (jeux de données) : **`\${nomDuParametre}`**

Le nom du paramètre ne doit comprendre aucun espace, caractère spécial ou accentué.

- Un paramètre peut être ajouté dans le bloc « Action », « Résultat attendu » ou « Prérequis » du cas de test.
- Il est possible de renseigner autant de paramètres que souhaité.
- Les paramètres renseignés dans les pas de test seront automatiquement reportés dans le tableau des paramètres de l'onglet 'Paramètres'.

- 2 Onglet 'Paramètre et jeux de données' : il est possible d'ajouter de nouveaux paramètres directement depuis cet onglet mais ils ne seront pas utilisés s'ils ne sont pas présents dans le prérequis ou les pas de test du cas de test.

- Il est donc recommandé de renseigner les paramètres directement dans l'onglet 'Pas de test' afin qu'ils soient automatiquement récupérés dans l'onglet "Paramètres".
- Pour ajouter un jeu de données, cliquer sur le bouton [+] (Ajouter un jeu de données) et renseigner la valeur qui remplacera le ou les paramètres.

Paramètres et jeux de données

+ Ajouter un jeu de données

NOM ↑		
	navigateur ...	+
Chrome	Chrome	🗑
Firefox	Firefox	🗑
IE	Internet Explorer	🗑

Lors de l'ajout du cas de test au plan d'exécution :

- Le paramètre sera remplacé par la valeur du jeu de données
- Il sera créé autant de cas de test que de jeux de données

11	<u>MAL_Dem...</u>		MAL1	<u>Vérifier la livraison des...</u>		Cappuccino -	
12	<u>MAL_Dem...</u>		MAL1	<u>Vérifier la livraison des...</u>		Espresso -	
13	<u>MAL_Dem...</u>		MAL1	<u>Vérifier la livraison des...</u>		Lungo -	

- Les jeux de données doivent être anticipés et conçus en phase de conception.
- Si possible, les jeux de données doivent être stables et réutilisables
- Un jeu de données spécifique doit être privilégié pour chaque cas de test afin d'éviter les faux positifs.



Variabilisation de cas de test

Créer un cas de test vérifiant les comportements suivants en utilisant les paramètres : $\${...}$:

- Il est possible de créer un élève avec un prénom composé
- Il est impossible de créer un élève avec un prénom contenant des chiffres

Méthode :

1. Créer un cas de test intitulé « Créer un élève »
2. Ecrire les pas de test avec les $\${Parametres}$.

Astuce : Imaginer un texte à trous (les trous étant les paramètres) qui conviendrait aux 2 situations

3. Dans l'onglet Paramètres, donner des valeurs aux paramètres dans le tableau des jeux de données



Variabilisation de cas de test

Créer un cas de test vérifiant les comportements suivants en utilisant les paramètres : $\${...}$:

- Il est possible de créer un élève avec un prénom composé
- Il est impossible de créer un élève avec un prénom contenant des chiffres

Ajouter des paramètres dans les blocs
"Actions" et "Résultat attendu"

Prérequis et Pas de test

► PREREQUIS :

	ACTION	RÉSULTAT ATTENDU
1	Se connecter en tant qu'administrateur à l'interface de création d'un élève	Le formulaire de création s'affiche
2	Dans le champ <i>prénom</i> , écrire $\${prenom}$, puis cliquer sur [Enregistrer]	Le message $\${conf}$ s'affiche: $\${contenu}$

Ajouter des jeux de données
dans l'onglet 'Paramètres'

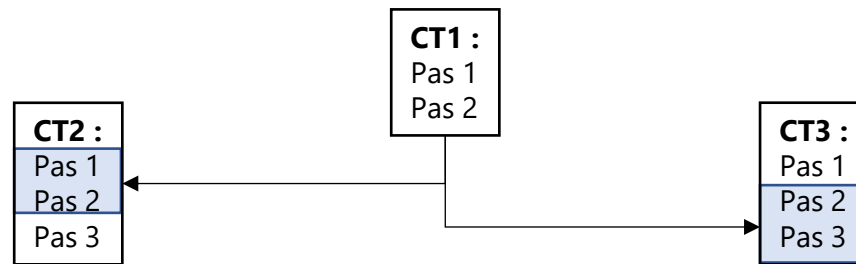
Paramètres et jeux de données

+ Ajouter un jeu de données

NOM ↑	conf ...	prenom ...	contenu ...	
Prénom avec chiffre	bloquant	J4N	Le prénom ne peut pas contenir de chiffre	+
Prénom composé	de configuration	Anne-Sophie	Nouvel élève enregistré	+

Onglet «Pas de test » :

- Le mécanisme d'**appels de tests** (« calls ») dans un cas de test permet la construction de bibliothèques modulaires de cas de test.
- Lors de l'exécution, les pas de test du cas de test appelé sont vu comme des pas de test dans le cas de test appelant.



- Ce mécanisme peut également être utilisé pour des tests de bout en bout faisant appel au patrimoine de tests de différents projets.
- Les cas de tests appelants sont listés dans le bloc « Cas de test appelé par » dans l'onglet 'Informations' du cas de test appelé.